

TECHNICAL ASSESSMENT

Distributed Observability Agent

Level: Senior / Mid-level Developer Duration: 4 – 6 hours

Design and implement a production-grade telemetry collection system that auto-instruments Node.js and Python services, stores traces and metrics in PostgreSQL, and exposes query results via a FastAPI JSON API — with data refreshed every 60 seconds.

01 TECHNOLOGY STACK

Go	Telemetry agent + OTLP collector
OpenTelemetry	Auto-instrumentation — traces, metrics, logs
Node.js	Target service — OTel JS SDK
Python	Target service — opentelemetry-sdk
PostgreSQL	Telemetry store — direct API connection

02 CORE REQUIREMENTS

- 01. Go Agent** — Build a Go binary that acts as an OTLP receiver (gRPC or HTTP). Accept spans, metrics, and logs from instrumented services, batch them in memory, and flush to PostgreSQL every 60 seconds using a ticker goroutine with graceful shutdown.
- 02. Node.js Auto-Instrumentation** — Provide a sample Express app instrumented via `@opentelemetry/auto-instrumentations-node`. Configure the OTLP exporter to send to the Go agent. No manual span creation required — demonstrate SDK bootstrap only.
- 03. Python Auto-Instrumentation** — Provide a sample FastAPI or Flask app instrumented via `opentelemetry-instrument` CLI or SDK bootstrap. Export to the same Go agent endpoint via OTLP/gRPC.
- 04. PostgreSQL Schema** — Design at minimum two tables: spans (`trace_id`, `span_id`, `service_name`, `operation`, `duration_ms`, `started_at`, attributes JSONB) and metrics (`service_name`, `metric_name`, `value`, `unit`, `recorded_at`). Provide a `schema.sql` migration file.
- 05. FastAPI JSON API** — Connect directly to PostgreSQL (`asyncpg` or `psycopg2`). Expose: `GET /spans` (filterable by `service_name`, time range), `GET /metrics` (latest values per service), and `GET /health`. All responses must be JSON with Pydantic response models.
- 06. 60-Second Collection Cycle** — The Go agent must not write on every received event. Accumulate data in a thread-safe buffer and commit one batch transaction per minute. Demonstrate this with a test or log output.

03 BONUS CHALLENGES

- B1 Sampling Strategy** — Implement probabilistic or tail-based sampling in the Go agent (e.g., drop spans < 5 ms). Expose sampling rate as a configurable environment variable.
- B2 Service Dependency Graph** — Add a `GET /graph` endpoint that queries trace parent-child relationships and returns an adjacency list JSON representing which services call which.
- B3 Prometheus Metrics Endpoint** — Expose `/metrics` from the Go agent in Prometheus text format so Grafana can scrape it alongside the PostgreSQL data.

04 EXPECTED SCHEMA (Reference)

```
CREATE TABLE spans (
  id          BIGSERIAL PRIMARY KEY,
  trace_id    TEXT NOT NULL,
  span_id     TEXT NOT NULL,
  parent_id   TEXT,
  service_name TEXT NOT NULL,
  operation    TEXT NOT NULL,
  duration_ms  NUMERIC(10,3),
  started_at   TIMESTAMPTZ NOT NULL,
  attributes   JSONB DEFAULT '{} '
);

CREATE TABLE metrics (
  id          BIGSERIAL PRIMARY KEY,
  service_name TEXT NOT NULL,
  metric_name  TEXT NOT NULL,
  value        DOUBLE PRECISION NOT NULL,
  unit         TEXT,
  recorded_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Indexes
CREATE INDEX ON spans(service_name, started_at);
CREATE INDEX ON spans USING GIN(attributes);
CREATE INDEX ON metrics(service_name, recorded_at DESC);
```

05 DELIVERABLES

Git Repository	Mono-repo or multi-module with clear README
schema.sql	Idempotent migration script
Test Evidence	Screenshot or logs showing 60-second batch flush
README.md	Architecture diagram + setup steps

06 EVALUATION RUBRIC

Evaluation Area	What We Look For	Weight
Go Agent Correctness	OTLP receiver works; goroutine safety; 60s ticker with proper shutdown	25 pts
OTel Instrumentation	Both Node.js and Python emit real traces/metrics without manual span creation	20 pts
PostgreSQL Design	Schema normalisation, JSONB, TIMESTAMPTZ, useful indexes	15 pts
FastAPI Quality	Async DB driver, Pydantic response models, filter params, OpenAPI docs present	15 pts
Code Quality	Error handling, logging, Go idioms, Python typing, separation of concerns	10 pts
README + Architecture	System diagram, data-flow description, clear run instructions	5 pts
Bonus Challenges	Any of B1-B3 correctly implemented	+15 pts

Passing score: 70 / 100 · Bonus points may offset any core area · Candidates must walk through architecture decisions in a follow-up discussion.